

Baseline_Balanced

July 14, 2026

1 Baseline balanceado del proyecto MIA

Este notebook implementa el primer experimento de mejora sobre el modelo baseline del proyecto MIA para la clasificación inicial de tickets de una mesa de ayuda TI universitaria.

El objetivo de este experimento es evaluar el impacto del uso del parámetro `class_weight='balanced'` en un modelo de Regresión Logística, manteniendo sin cambios el conjunto de datos, el preprocesamiento, la vectorización mediante TF-IDF y el esquema de evaluación utilizados en el baseline original.

El flujo desarrollado corresponde a la primera iteración de optimización del modelo:

Cargar datos → EDA básico → Limpiar texto → Vectorizar con TF-IDF → Entrenar Logistic Regression (`class_weight='balanced'`) → Evaluar con Classification Report y matriz de confusión → Comparar con el baseline.

1.1 1. Configuración inicial

En esta sección se cargan las librerías necesarias, se define la semilla aleatoria y se configuran las rutas de trabajo en Google Drive.

```
[1]: # Librerías estándar
import os
import re
import string
import warnings

# Manejo de datos
import numpy as np
import pandas as pd

# Visualización
import matplotlib.pyplot as plt
import seaborn as sns

# Machine Learning
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
```

```

        accuracy_score,
        classification_report,
        confusion_matrix
    )

    # Configuración general
    warnings.filterwarnings("ignore")
    RANDOM_STATE = 42

    plt.style.use("default")
    plt.rcParams["figure.figsize"] = (10, 6)

    print(" Configuración inicial completada.")

```

Configuración inicial completada.

1.2 2. Carga del dataset

En esta sección se conecta Google Drive y se carga el archivo original de tickets ubicado en la carpeta 01_data/raw.

```

[2]: # Montar Google Drive
from google.colab import drive
drive.mount('/content/drive')

# Ruta principal del proyecto
PROJECT_ROOT = "/content/drive/MyDrive/Proyecto_MIA"

# Ruta del dataset original
DATASET_PATH = f"{PROJECT_ROOT}/01_data/raw/tickets_raw_corte_01_2026-02-25.
↳xlsx"

# Cargar dataset
df = pd.read_excel(DATASET_PATH)

print(f"Dataset cargado correctamente.")
print(f"Registros: {df.shape[0]:,}")
print(f"Columnas : {df.shape[1]}")

```

Mounted at /content/drive
 Dataset cargado correctamente.
 Registros: 24,392
 Columnas : 42

1.3 3. EDA básico

En esta sección se realiza una revisión inicial del dataset para validar su estructura, identificar valores nulos y revisar la distribución de la variable objetivo Grupo.

```
[3]: # Dimensiones del dataset

print(f"Número de registros : {df.shape[0]:,}")
print(f"Número de columnas : {df.shape[1]:,}")

display(df.head())
```

Número de registros : 24,392
 Número de columnas : 42

	Fecha de registro	Código \
0	2023-02-08 17:33:06	REQ 2023-003487
1	2023-02-09 10:00:07	REQ 2023-003539
2	2023-02-14 12:57:06	REQ 2023-003967
3	2023-02-20 17:45:06	REQ 2023-004390
4	2023-02-22 11:18:06	REQ 2023-004477

	Título	Estado \
0	RV: Entrega de novedades SEGUNDO SEMESTRE 2022...	Cerrada
1	RE: Entrega de novedades SEGUNDO SEMESTRE 2022...	Cerrada
2	FORMULARIO 107	Cerrada
3	Formulario 107	Cerrada
4	Problemas en el servidor	Cerrada

	Fecha firma cierre	Acción a realizar	Fecha firma solución \
0	2025-09-11 09:32:55	Escalar a 2ª línea	2025-09-01 09:31:12
1	2025-09-11 09:32:55	Escalar a 2ª línea	2025-09-01 09:31:12
2	2025-09-11 09:32:55	Escalar a 2ª línea	2025-09-01 09:31:11
3	2025-09-11 09:32:54	Escalar a 2ª línea	2025-09-01 09:31:11
4	2025-09-11 09:32:54	Escalar a 2ª línea	2025-09-01 09:31:10

	Firma solución	Escalado	Tipo ...	SLA \
0	Tufiño Benalcázar, Carlos	Sí	Solicitud RRHH ...	SLA 24 Hrs RRHH
1	Tufiño Benalcázar, Carlos	Sí	Solicitud RRHH ...	SLA 24 Hrs RRHH
2	Tufiño Benalcázar, Carlos	Sí	Solicitud RRHH ...	SLA 24 Hrs RRHH
3	Tufiño Benalcázar, Carlos	Sí	Solicitud RRHH ...	SLA 24 Hrs RRHH
4	Tufiño Benalcázar, Carlos	Sí	Solicitud RRHH ...	SLA 24 Hrs RRHH

	Solución para el usuario Técnico que realiza última comunicación a cliente \
0	Solucionado NaN
1	Solucionado NaN
2	Solucionado NaN
3	Solucionado NaN
4	Solucionado NaN

	Tiempo 1ª respuesta	Tiempo atención (horas / minutos)	Tiempo de atención \
0	5815.517578	0h 00m	0.00
1	5813.517187	1h 24m	1.40

2	5783.567188	5h 27m	5.45
3	5743.500000	3h 00m	3.00
4	5731.217318	0h 08m	0.13

	Tiempo de cierre	Tiempo de cierre (horas / minutos)	Tiempo de resolución \
0	5887.534241	5887h 32m	5815.517578
1	5885.533850	5885h 32m	5813.517187
2	5855.583851	5855h 35m	5783.567188
3	5815.516663	5815h 31m	5743.500000
4	5803.233980	5803h 14m	5731.217318

	Última comunicación a cliente
0	NaT
1	NaT
2	NaT
3	NaT
4	NaT

[5 rows x 42 columns]

[4]: *# Información general del dataset*

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 24392 entries, 0 to 24391
Data columns (total 42 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Fecha de registro                    24392 non-null  datetime64[ns]
1   Código                               24392 non-null  object
2   Título                               24392 non-null  object
3   Estado                               24392 non-null  object
4   Fecha firma cierre                   14364 non-null  datetime64[ns]
5   Acción a realizar                    24392 non-null  object
6   Fecha firma solución                 16849 non-null  datetime64[ns]
7   Firma solución                       16849 non-null  object
8   Escalado                             24392 non-null  object
9   Tipo                                 24390 non-null  object
10  Responsable última modificación      24372 non-null  object
11  Grupo                                17237 non-null  object
12  Caducada                             24392 non-null  object
13  Asignaciones grupo actual            17255 non-null  float64
14  Categoría                             20695 non-null  object
15  Técnico de 2ª línea                 17152 non-null  object
```

16	Cerrada / Revocada por	18858 non-null	object
17	Comentarios de cierre	14203 non-null	object
18	Comentarios de la investigación	2 non-null	object
19	Descripción	24076 non-null	object
20	Enviado mail	24392 non-null	object
21	Fecha estimada resolución	20058 non-null	
datetime64[ns]			
22	Fecha estimada respuesta	123 non-null	
datetime64[ns]			
23	Fecha última modificación	24372 non-null	
datetime64[ns]			
24	Horas dedicadas	24392 non-null	float64
25	Firma cierre / revocación solución	14364 non-null	object
26	Incidencia / Petición principal	11 non-null	object
27	Intentos de solución	24392 non-null	int64
28	Origen	24392 non-null	object
29	Reasignaciones grupo	24392 non-null	int64
30	Reasignaciones totales	24392 non-null	int64
31	Servicio 1ª línea	13553 non-null	object
32	SLA	20059 non-null	object
33	Solución para el usuario	16812 non-null	object
34	Técnico que realiza última comunicación a cliente	4176 non-null	object
35	Tiempo 1ª respuesta	24144 non-null	float64
36	Tiempo atención (horas / minutos)	22360 non-null	object
37	Tiempo de atención	22360 non-null	float64
38	Tiempo de cierre	23525 non-null	float64
39	Tiempo de cierre (horas / minutos)	23525 non-null	object
40	Tiempo de resolución	23971 non-null	float64
41	Última comunicación a cliente	4176 non-null	
datetime64[ns]			
dtypes: datetime64[ns](7), float64(6), int64(3), object(26)			
memory usage: 7.8+ MB			

1.3.1 Análisis preliminar del conjunto de datos

La inspección inicial muestra que el dataset está compuesto por **24.392 tickets** y **42 variables** de diferentes tipos (texto, fechas y variables numéricas).

De acuerdo con el objetivo del proyecto y con las validaciones realizadas con el personal de la mesa de ayuda, el modelo baseline utilizará como variable objetivo el campo **Grupo**, ya que representa la asignación inicial del grupo resolutorio en la mayoría de los casos.

Asimismo, para evitar *Data Leakage*, en esta primera versión no se utilizarán variables que contienen información generada después de la clasificación inicial del ticket, como tiempos de resolución, tiempos de cierre, solución aplicada o información completada posteriormente durante el proceso de atención.

```
[5]: # Distribución de tickets por estado
```

```

estado_counts = (
    df["Estado"]
    .value_counts(dropna=False)
    .reset_index()
)

estado_counts.columns = ["Estado", "Cantidad"]

display(estado_counts)

print(f"\nNúmero de estados encontrados: {df['Estado'].nunique()}")

```

	Estado	Cantidad
0	Cerrada	18855
1	Rechazada	4687
2	Resuelta	445
3	Asignada a un grupo	388
4	Nueva	17

Número de estados encontrados: 5

1.3.2 Selección de los tickets para el entrenamiento

El análisis de la variable **Estado** muestra que la mayoría de los tickets corresponden a los estados **Cerrada** y **Rechazada**.

De acuerdo con la validación realizada con el personal de la Dirección de Tecnologías, estos estados representan casos con una resolución definitiva. En cambio, los tickets en estado **Resuelta** pueden volver a abrirse si el usuario no está conforme con la solución, mientras que los estados **Nueva** y **Asignada a un grupo** corresponden a casos que aún se encuentran en proceso.

Por este motivo, el modelo baseline será entrenado únicamente con tickets en estado **Cerrada** y **Rechazada**, garantizando que el conjunto de entrenamiento represente decisiones operativas consolidadas y evitando incorporar registros cuyo flujo aún no ha finalizado.

[6]: *# Valores nulos en las variables relevantes del proyecto*

```

columnas_interes = ["Título", "Descripción", "Grupo"]

nulos = (
    df[columnas_interes]
    .isnull()
    .sum()
    .reset_index()
)

nulos.columns = ["Variable", "Valores nulos"]
nulos["Porcentaje (%)"] = (

```

```

nulos["Valores nulos"] / len(df) * 100
).round(2)

display(nulos)

```

	Variable	Valores nulos	Porcentaje (%)
0	Título	0	0.00
1	Descripción	316	1.30
2	Grupo	7155	29.33

```
[7]: # Distribución de la variable objetivo (Grupo)
```

```

grupo_counts = (
    df["Grupo"]
    .value_counts()
    .sort_values(ascending=False)
)

plt.figure(figsize=(12,6))
grupo_counts.plot(kind="bar")

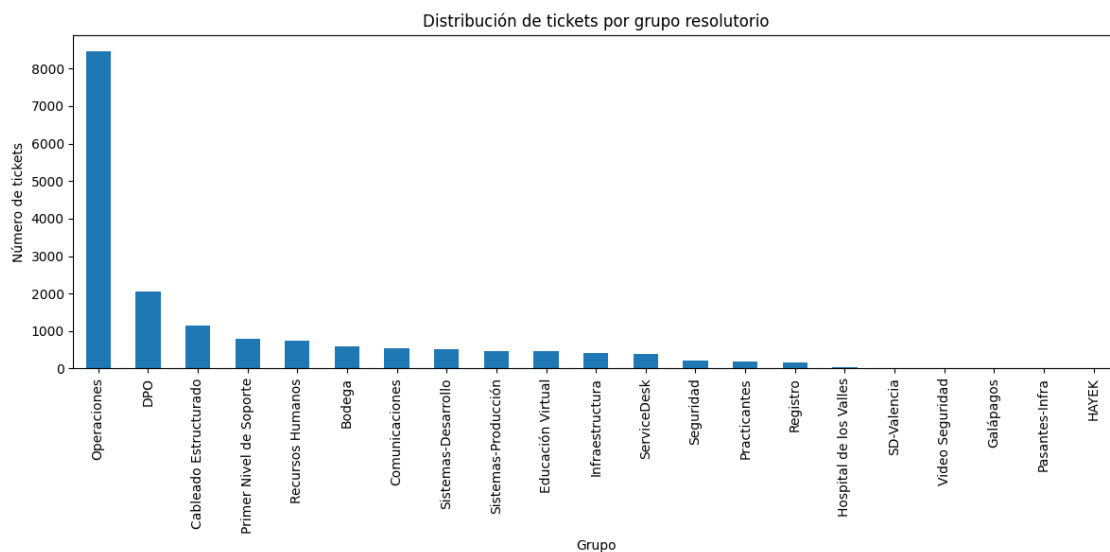
plt.title("Distribución de tickets por grupo resolutorio")
plt.xlabel("Grupo")
plt.ylabel("Número de tickets")

plt.xticks(rotation=90)

plt.tight_layout()
plt.show()

print(f"Número de grupos resolutorios: {grupo_counts.shape[0]}")

```



Número de grupos resolutorios: 21

1.3.3 Conclusiones del análisis exploratorio

El análisis exploratorio permitió identificar que el problema corresponde a una tarea de clasificación multiclase con **21 grupos resolutorios**. Asimismo, se verificó la existencia de valores nulos en la variable objetivo (*Grupo*), los cuales deberán tratarse durante la preparación de los datos.

Con base en este análisis, la siguiente etapa consistirá en construir el conjunto de datos que será utilizado para entrenar el modelo baseline.

1.4 4. Preparación de los datos

En esta sección se construirá el conjunto de datos que será utilizado para entrenar el modelo baseline. Para ello se aplicarán los filtros y transformaciones definidos durante el análisis exploratorio, garantizando que el modelo utilice únicamente información disponible al momento de la clasificación inicial del ticket.

```
[8]: # Construcción del dataset para Machine Learning

df_ml = df[
    (df["Estado"].isin(["Cerrada", "Rechazada"])) &
    (df["Grupo"].notna())
].copy()

print(f"Registros del dataset original : {len(df):,}")
print(f"Registros para Machine Learning: {len(df_ml):,}")
print(f"Registros descartados           : {len(df) - len(df_ml):,}")
```

```
Registros del dataset original : 24,392
Registros para Machine Learning: 16,404
Registros descartados          : 7,988
```

```
[9]: # Verificar valores nulos en las variables utilizadas por el modelo

print("Valores nulos en el dataset de Machine Learning:\n")

print(f"Título       : {df_ml['Título'].isnull().sum()}")
print(f"Descripción  : {df_ml['Descripción'].isnull().sum()}")
print(f"Grupo        : {df_ml['Grupo'].isnull().sum()}")
```

Valores nulos en el dataset de Machine Learning:

```
Título       : 0
Descripción  : 288
Grupo        : 0
```



```
[10]: # Eliminar registros sin descripción

df_ml = df_ml.dropna(subset=["Descripción"]).copy()

print(f"Registros disponibles para entrenamiento: {len(df_ml):,}")
```

Registros disponibles para entrenamiento: 16,116

1.4.1 Construcción del texto de entrada

El modelo baseline utilizará como entrada una única representación textual construida a partir de la concatenación del **Título** y la **Descripción** del ticket.

Esta estrategia permite aprovechar tanto el resumen inicial del incidente como el detalle proporcionado por el usuario, generando una representación más completa para el proceso de clasificación.

```
[11]: # Construir el texto de entrada para el modelo

df_ml["Texto"] = (
    df_ml["Título"].str.strip() + " " +
    df_ml["Descripción"].str.strip()
)

# Eliminar espacios redundantes
df_ml["Texto"] = df_ml["Texto"].str.replace(r"\s+", " ", regex=True).str.strip()

print("Ejemplos del texto de entrada:\n")

display(df_ml[["Título", "Descripción", "Texto"]].head())
```

Ejemplos del texto de entrada:

	Título \
0	RV: Entrega de novedades SEGUNDO SEMESTRE 2022...
1	RE: Entrega de novedades SEGUNDO SEMESTRE 2022...
2	FORMULARIO 107
3	Formulario 107
4	Problemas en el servidor

	Descripción \
0	Buen día,\n\nEnvío la aprobación de Paolo Monc...
1	\n\nBuen día,\n\nEnvío la aprobación de Paolo ...
2	Buenas tardes les pido de su gentileza me ayud...
3	Buenas tardes, espero que estén bien y que est...
4	Saludos,\n\nSoy Alvaro Villacis, profesor a ti...

	Texto
0	RV: Entrega de novedades SEGUNDO SEMESTRE 2022...

```

1 RE: Entrega de novedades SEGUNDO SEMESTRE 2022...
2 FORMULARIO 107 Buenas tardes les pido de su ge...
3 Formulario 107 Buenas tardes, espero que estén...
4 Problemas en el servidor Saludos, Soy Alvaro V...

```

1.4.2 Limpieza del texto

Como parte del baseline, se aplicará un preprocesamiento mínimo al texto de entrada. Este proceso incluye la conversión a minúsculas, la eliminación de signos de puntuación y la normalización de espacios en blanco.

No se aplicarán técnicas adicionales como eliminación de stopwords, stemming o lematización, ya que estas formarán parte de los experimentos posteriores del proyecto.

```

[12]: # Función para la limpieza básica del texto

def limpiar_texto(texto):
    """
    Realiza la limpieza básica del texto para el modelo baseline.
    """

    # Convertir a minúsculas
    texto = texto.lower()

    # Eliminar signos de puntuación
    texto = texto.translate(str.maketrans("", "", string.punctuation))

    # Eliminar espacios múltiples
    texto = re.sub(r"\s+", " ", texto)

    # Eliminar espacios al inicio y al final
    texto = texto.strip()

    return texto

# Aplicar limpieza al texto
df_ml["Texto_Limpio"] = df_ml["Texto"].apply(limpiar_texto)

print("Ejemplos del texto limpio:\n")

display(df_ml[["Texto", "Texto_Limpio"]].head())

```

Ejemplos del texto limpio:

	Texto	\
0	RV: Entrega de novedades SEGUNDO SEMESTRE 2022...	
1	RE: Entrega de novedades SEGUNDO SEMESTRE 2022...	

```

2 FORMULARIO 107 Buenas tardes les pido de su ge...
3 Formulario 107 Buenas tardes, espero que estén...
4 Problemas en el servidor Saludos, Soy Alvaro V...

```

```

                                Texto_Limpio
0 rv entrega de novedades segundo semestre 20222...
1 re entrega de novedades segundo semestre 20222...
2 formulario 107 buenas tardes les pido de su ge...
3 formulario 107 buenas tardes espero que estén ...
4 problemas en el servidor saludos soy alvaro vi...

```

```
[14]: # Identificar grupos con muy pocos registros
```

```

grupo_frecuencia = y.value_counts()

display(grupo_frecuencia[grupo_frecuencia <= 5].sort_values())

```

```

Grupo
HAYEK          1
Pasantes-Infra 2
Name: count, dtype: int64

```

1.4.3 Tratamiento de clases con muy baja representación

Durante la preparación del conjunto de datos se identificó un grupo resolutorio con un único registro histórico. Debido a que una clase con un solo ejemplo no permite realizar una partición estratificada ni proporciona información suficiente para el entrenamiento de un modelo supervisado, dicho registro fue excluido del dataset utilizado para el baseline.

Esta decisión busca garantizar la reproducibilidad del proceso de entrenamiento sin afectar de forma significativa el conjunto de datos disponible.

```
[15]: # Eliminar clases con un único registro
```

```

grupo_frecuencia = df_ml["Grupo"].value_counts()

grupos_validos = grupo_frecuencia[grupo_frecuencia >= 3].index

df_ml = df_ml[df_ml["Grupo"].isin(grupos_validos)].copy()

print(f"Registros disponibles para entrenamiento: {len(df_ml):,}")
print(f"Número de grupos resolutorios: {df_ml['Grupo'].nunique()}")

```

```

Registros disponibles para entrenamiento: 16,113
Número de grupos resolutorios: 19

```

```
[16]: # Variables de entrada (X) y variable objetivo (y)
```

```
X = df_ml["Texto_Limpio"]
```

```

y = df_ml["Grupo"]

print(f"Número de documentos: {len(X):,}")
print(f"Número de etiquetas : {len(y):,}")
print(f"Número de grupos    : {y.nunique():,}")

```

```

Número de documentos: 16,113
Número de etiquetas : 16,113
Número de grupos    : 19

```

```

[17]: # División del conjunto de datos
      # 70% Entrenamiento
      # 15% Validación
      # 15% Prueba

      # Primera división (70% entrenamiento, 30% temporal)
      X_train, X_temp, y_train, y_temp = train_test_split(
          X,
          y,
          test_size=0.30,
          random_state=RANDOM_STATE,
          stratify=y
      )

      # Segunda división (15% validación, 15% prueba)
      X_val, X_test, y_val, y_test = train_test_split(
          X_temp,
          y_temp,
          test_size=0.50,
          random_state=RANDOM_STATE,
          stratify=y_temp
      )

      print("Distribución de los conjuntos:\n")

      print(f"Entrenamiento : {len(X_train):,} registros")
      print(f"Validación    : {len(X_val):,} registros")
      print(f"Prueba          : {len(X_test):,} registros")

```

Distribución de los conjuntos:

```

Entrenamiento : 11,279 registros
Validación    : 2,417 registros
Prueba        : 2,417 registros

```

1.5 5. Vectorización mediante TF-IDF

Una vez preparado el conjunto de datos, se procede a dividirlo en los conjuntos de entrenamiento, validación y prueba, preservando la distribución de la variable objetivo mediante una partición estratificada.

Posteriormente, se aplica la técnica **TF-IDF (Term Frequency–Inverse Document Frequency)** para transformar el texto en una representación numérica que pueda ser utilizada por el modelo de Regresión Logística. El vocabulario del vectorizador se construye únicamente con el conjunto de entrenamiento para evitar *Data Leakage*, mientras que los conjuntos de validación y prueba son transformados utilizando ese mismo vocabulario.

```
[18]: # Crear el vectorizador TF-IDF

tfidf = TfidfVectorizer()

# Aprender el vocabulario únicamente del conjunto de entrenamiento
X_train_tfidf = tfidf.fit_transform(X_train)

# Transformar validación y prueba utilizando el mismo vocabulario
X_val_tfidf = tfidf.transform(X_val)
X_test_tfidf = tfidf.transform(X_test)

print("Vectorización completada.\n")

print(f"Tamaño matriz Train      : {X_train_tfidf.shape}")
print(f"Tamaño matriz Validación : {X_val_tfidf.shape}")
print(f"Tamaño matriz Prueba      : {X_test_tfidf.shape}")

print(f"\nNúmero de términos del vocabulario: {len(tfidf.vocabulary_):,}")
```

Vectorización completada.

```
Tamaño matriz Train      : (11279, 38108)
Tamaño matriz Validación : (2417, 38108)
Tamaño matriz Prueba      : (2417, 38108)
```

Número de términos del vocabulario: 38,108

1.6 6. Entrenamiento del modelo baseline

En esta sección se entrena el modelo baseline utilizando **Regresión Logística**, tomando como entrada la representación vectorial generada mediante TF-IDF.

Este modelo servirá como punto de referencia para comparar futuras mejoras del proyecto, tanto en el procesamiento del texto como en la incorporación de otros algoritmos de clasificación.

```
[19]: # Entrenamiento del modelo baseline balanceado

modelo = LogisticRegression()
```

```

        class_weight="balanced",
        random_state=RANDOM_STATE,
        max_iter=1000
    )

    modelo.fit(X_train_tfidf, y_train)

    print("Modelo balanceado entrenado correctamente.")

```

Modelo balanceado entrenado correctamente.

1.7 7. Evaluación del modelo baseline

En esta sección se evalúa el desempeño del modelo baseline utilizando el conjunto de prueba, el cual no participó en el entrenamiento del modelo.

La evaluación se realizará mediante métricas de clasificación y una matriz de confusión, permitiendo establecer una línea base objetiva para comparar futuras mejoras del proyecto.

[20]: *# Generar predicciones sobre el conjunto de prueba*

```

y_pred = modelo.predict(X_test_tfidf)

print("Predicciones generadas correctamente.")
print(f"Número de predicciones: {len(y_pred):,}")

```

Predicciones generadas correctamente.

Número de predicciones: 2,417

[23]: *# Evaluación del modelo baseline*

```

accuracy = accuracy_score(y_test, y_pred)

print(f"Accuracy: {accuracy:.4f}\n")

print("Classification Report:\n")

print(classification_report(y_test, y_pred))

```

Accuracy: 0.6909

Classification Report:

	precision	recall	f1-score	support
Bodega	0.47	0.78	0.59	68
Cableado Estructurado	0.50	0.87	0.63	158
Comunicaciones	0.55	0.75	0.64	73
DPO	1.00	0.95	0.98	288

Educación Virtual	0.72	0.87	0.79	68
Galápagos	0.00	0.00	0.00	1
Hospital de los Valles	0.13	0.67	0.22	6
Infraestructura	0.24	0.41	0.30	58
Operaciones	0.97	0.58	0.73	1194
Practicantes	0.08	0.38	0.13	26
Primer Nivel de Soporte	0.48	0.54	0.51	109
Recursos Humanos	0.75	0.92	0.83	110
Registro	0.51	0.76	0.61	25
SD-Valencia	0.20	0.33	0.25	3
Seguridad	0.39	0.61	0.47	31
ServiceDesk	0.70	0.71	0.71	56
Sistemas-Desarrollo	0.64	0.81	0.72	73
Sistemas-Producción	0.70	0.88	0.78	69
Video Seguridad	0.00	0.00	0.00	1
accuracy			0.69	2417
macro avg	0.48	0.62	0.52	2417
weighted avg	0.81	0.69	0.72	2417

```
[25]: # Matriz de confusión

cm = confusion_matrix(y_test, y_pred, labels=modelo.classes_)

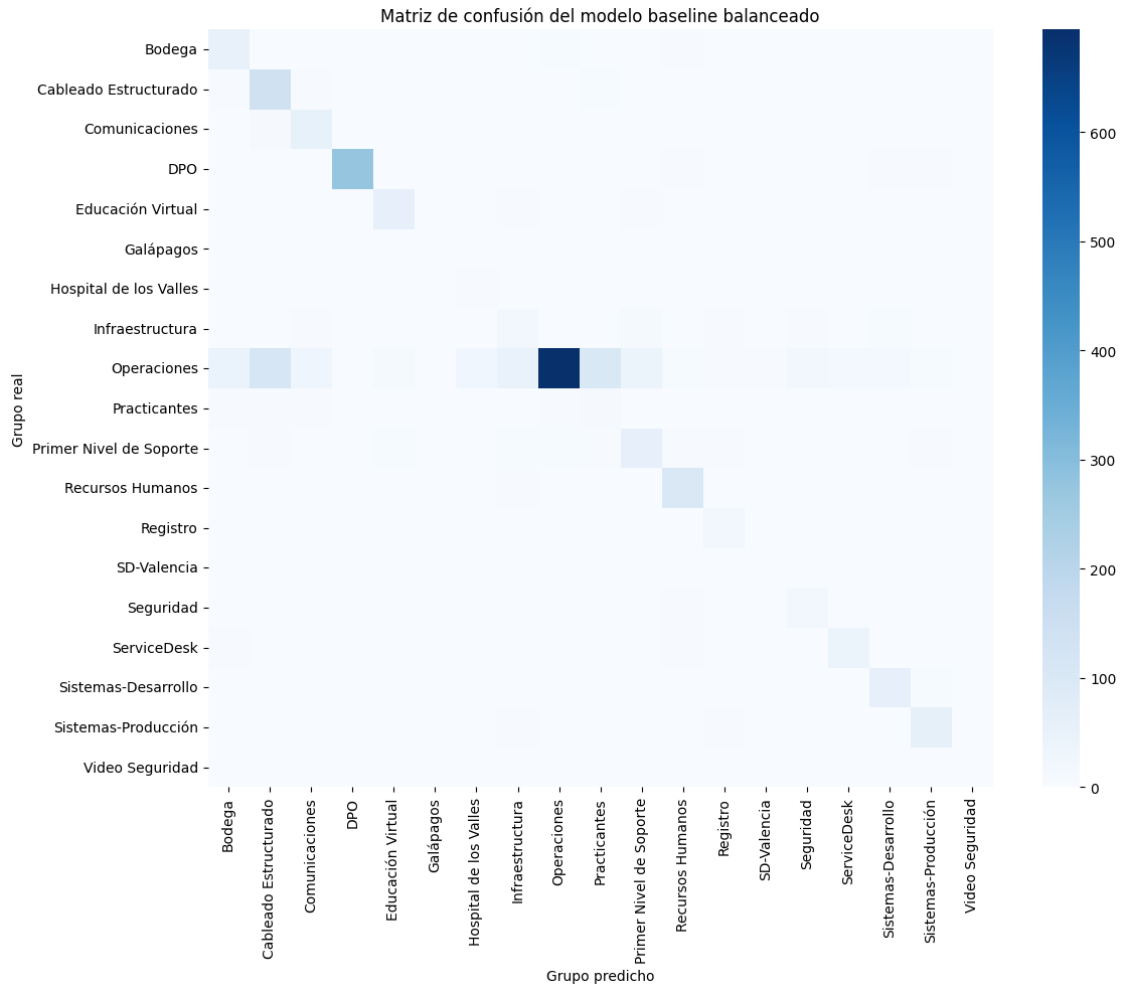
plt.figure(figsize=(12, 10))

sns.heatmap(
    cm,
    cmap="Blues",
    xticklabels=modelo.classes_,
    yticklabels=modelo.classes_,
    cbar=True
)

plt.title("Matriz de confusión del modelo baseline balanceado")
plt.xlabel("Grupo predicho")
plt.ylabel("Grupo real")

plt.xticks(rotation=90)
plt.yticks(rotation=0)

plt.tight_layout()
plt.show()
```



2 8. Comparación entre el baseline y el baseline balanceado

Como parte del proceso de mejora del modelo baseline, se evaluó el efecto del parámetro `class_weight='balanced'` sobre el desempeño del clasificador de Regresión Logística. A continuación, se comparan las principales métricas obtenidas en ambos experimentos.

```
[26]: import pandas as pd

comparacion = pd.DataFrame({
    "Métrica": [
        "Accuracy",
        "Precision Macro",
        "Recall Macro",
        "F1 Macro",
        "Precision Weighted",
        "Recall Weighted",
    ]
})
```



```

        "F1 Weighted"
    ],
    "Baseline": [
        0.7894,
        0.57,
        0.44,
        0.45,
        0.79,
        0.79,
        0.77
    ],
    "Baseline Balanceado": [
        0.6909,
        0.48,
        0.62,
        0.52,
        0.81,
        0.69,
        0.72
    ]
}

comparacion["Diferencia"] = (
    comparacion["Baseline Balanceado"] -
    comparacion["Baseline"]
)

comparacion

```

```

[26]:
      Métrica  Baseline  Baseline Balanceado  Diferencia
0      Accuracy      0.7894      0.6909      -0.0985
1  Precision Macro      0.5700      0.4800      -0.0900
2      Recall Macro      0.4400      0.6200      0.1800
3        F1 Macro      0.4500      0.5200      0.0700
4  Precision Weighted      0.7900      0.8100      0.0200
5      Recall Weighted      0.7900      0.6900      -0.1000
6        F1 Weighted      0.7700      0.7200      -0.0500

```

2.0.1 Análisis

La incorporación del parámetro `class_weight='balanced'` produjo un comportamiento mixto en el modelo. Por una parte, el F1-Score Macro aumentó de 0.45 a 0.52, indicando una mejor capacidad para clasificar los grupos resolutorios con menor representación. Sin embargo, esta mejora se obtuvo a costa de una disminución significativa del Accuracy (78,94 % a 69,09 %) y del F1-Score Weighted, evidenciando una pérdida de desempeño global del clasificador.

Estos resultados muestran que el balanceo de clases mejora la atención sobre las clases minoritarias, pero introduce un deterioro importante en las clases con mayor cantidad de ejemplos, especialmente

en el grupo Operaciones.

3 9. Conclusiones

El experimento realizado permitió evaluar el impacto del parámetro `class_weight='balanced'` sobre el modelo baseline de clasificación de tickets. Aunque se observó una mejora en el F1-Score Macro, el deterioro del Accuracy y del F1-Score Weighted indica que esta estrategia, por sí sola, no constituye la mejor alternativa para optimizar el desempeño general del clasificador.

A partir de estos resultados, las siguientes iteraciones del proyecto se orientarán a mejorar la calidad de la información textual utilizada como entrada del modelo mediante la implementación de un módulo de evaluación y estandarización de títulos basado en técnicas de PLN apoyadas en reglas, diccionarios y expresiones regulares. Se espera que este enfoque incremente la consistencia semántica de los tickets y contribuya a mejorar la clasificación de los grupos resolutorios sin sacrificar el desempeño global del modelo.